PROBLEM 1 · SUPERVISED LEARNING · STUDENT BRIEF

Predicting 30-day hospital *readmission* risk.

Build the real thing — from messy clinical data all the way to a live, monitored, governed service a care team could actually trust. Read this whole page before you write any code.

~100k real patient encounters

End-to-end data → deploy → monitor → govern

80% of the grade is around the model

01 — YOUR BRIEF

The job, in *one paragraph.*

You are the data science team for a regional hospital network. Too many diabetic patients are readmitted within 30 days of discharge — costly for the hospital, bad for patients. Your task: at the moment of discharge, predict each patient's risk of 30-day readmission, so care teams can give the highest-risk patients extra follow-up. But a model in a notebook helps no one — you must take it all the way to a running, monitored, governed service.

THE POINT OF THIS PROJECT

You are not just training a model. You are building **and defending** a complete, production-style ML system. The modeling is roughly 20% of your grade; the other 80% is everything around it.

Why this is harder than it looks

- **The data is imbalanced.** Only ~11% of patients return within 30 days — so a model that always says "won't return" scores ~89% accuracy and is useless. Accuracy is a trap.
- **The strongest signals aren't obvious.** Number of prior inpatient visits often matters more than a scary-looking lab value.
- **The data is messy.** Missing values coded as "?", columns that are almost entirely empty, high-cardinality medical codes.
- **It's a regulated, human domain.** Fairness, explainability, and auditability are part of the deliverable — not extras.

02 — THE DATA

Where to get it & *what's inside.*

PRIMARY SOURCE · UCI ML REPOSITORY

Diabetes 130-US Hospitals (1999–2008) ↗

archive.ics.uci.edu/dataset/296

CONVENIENT CSV MIRROR · KAGGLE

Diabetes dataset (ready-to-load CSV) ↗

kaggle.com/datasets/brandao/diabetes

OPTIONAL · FAIRLEARN (PRE-BINARIZED, FAIRNESS-READY)

fetch_diabetes_hospital() ↗

fairlearn.org · diabetes_hospital_data

101,766
hospital encounters (one row = one stay)

~50
features: demographics, labs, meds, diagnoses, prior visits

~11%
readmitted within 30 days — heavily imbalanced

3
target classes: <30, >30, NO

~97%
of the "weight" column is missing

130
hospitals, over 10 years (1999–2008)

Known traps — you'll lose marks if you miss these

- **Missing values are coded as "?"**, not blanks. Untreated, your model reads "?" as a real category.
- **"weight" is ~97% empty**; payer_code and medical_specialty are heavily missing. Decide and justify: drop or impute?

- **A missing A1c is not a healthy A1c.** "Not measured" is information — don't fill it with a normal value.
- **The same patient appears in multiple rows.** Split randomly by row and the same patient lands in train and test — that's leakage. Split by patient ID.
- **Choose your target deliberately.** Most teams use "<30" vs everything else. State your choice and why.

03 — WHAT YOU MUST DO

Stage by *stage.*

Work through these in order. The checklist is the minimum, not the ceiling.

STAGE 1 Data Engineering & Exploration

1. Profile the raw data: types, missingness per column, class balance.
2. Build a reproducible cleaning pipeline (a script/function, not loose cells) that handles "?" and decides on high-missing columns.
3. Define your prediction target explicitly and document it.
4. Version your data & cleaning steps (DVC, or a clearly documented re-runnable script).

STAGE 2 Feature Engineering

1. Group high-cardinality ICD-9 diagnoses into clinical buckets (circulatory, respiratory, diabetes…).
2. Encode categoricals; engineer features like total prior visits, medication-change count, number of meds.
3. Keep a written record of every feature and why it exists.

STAGE 3

Modeling & Evaluation

1. Split **by patient ID** into train/val/test. The test set is touched once, at the end.
2. Build a simple baseline first (logistic regression + class weighting) — the bar to beat.
3. Train a stronger model (XGBoost / LightGBM); tune with cross-validation; handle imbalance.
4. Evaluate with PR-AUC, recall at fixed precision, and calibration. **Accuracy is not your headline metric.**
5. Choose & justify an operating threshold tied to the real cost trade-off.
6. Track every experiment in MLflow.

STAGE 4

Package & Deploy

1. Wrap the model in a FastAPI service returning a risk score + top contributing factors.
2. Containerize with Docker; run locally with Docker Compose; reproducible build.
3. Deploy to a free-tier host so it's reachable at a URL.
4. Document a rollback plan.

STAGE 5

Observability & Monitoring

1. **Service metrics:** latency, request count, error rate via Prometheus + a Grafana dashboard (and/or CloudWatch / Cloud Monitoring once deployed).
2. **Model metrics:** log every prediction; track the score/confidence distribution over time.
3. **Drift:** monitor data drift and prediction drift with Evidently, comparing live inputs to the training distribution; optionally use SageMaker Model Monitor or Vertex AI Model Monitoring.
4. **Alerting:** set at least one alert (e.g. error rate or drift threshold) that would notify a human.
5. **Retraining trigger:** define it concretely — e.g. "if drift score > X, or recall on new labelled data < Y, retrain."

STAGE 6

Governance & Re-evaluation

1. **Fairness:** audit performance across demographic groups (age, gender, race) with Fairlearn; report disparities, not just an average.
2. **Explainability:** SHAP for global feature importance and per-patient explanations the care team could read.
3. **Auditability:** log every scored request (inputs, score, model version, timestamp) so any decision can be traced later.
4. **Model card:** intended use, training data, performance, limitations, and fairness findings.
5. **Lineage & versioning:** which data + code + model version produced this prediction (MLflow registry).
6. **Human-in-the-loop:** state how low-confidence or borderline cases get a human review.

04 — TOOLS YOU CAN USE

All free, all *open source.*

STAGE	RECOMMENDED TOOLS
Data engineering	pandas / Polars, Great Expectations (validation), DVC (versioning)
Modeling	scikit-learn, XGBoost / LightGBM, MLflow (tracking + registry)
Serving	FastAPI, Uvicorn, Docker, Docker Compose
Deployment	Docker + a registry (Amazon ECR / Google Artifact Registry), then **AWS ECS Fargate (Express Mode) or Lambda**, or **GCP Cloud Run**
Observability	Prometheus + Grafana, Evidently AI (drift); cloud-native: AWS CloudWatch / GCP Cloud Monitoring; advanced: SageMaker Model Monitor / Vertex AI Model Monitoring
Governance	SHAP (explainability), Fairlearn (fairness), model-card template; audit logging via CloudWatch Logs / Cloud Logging; lineage via MLflow registry

You may use equivalent tools — but you must justify the choice and run everything within a free tier or locally.

05 — HOW TO DEPLOY (AWS & GCP)

From your laptop to a *live cloud service.*

You don't need to invent infrastructure. Build the container once, then deploy it to **AWS or GCP** (you'll do at least one; doing both earns bonus credit and teaches portability). The container is identical — only the hosting target changes.

1. Save your trained model (joblib/pickle) and write a **FastAPI** app with a **/predict** endpoint returning a JSON risk score + top contributing factors.
2. Write a **Dockerfile**; build the image, run the container, hit the endpoint locally.
3. Use **Docker Compose** to run your API alongside Prometheus + Grafana — your whole stack on one machine before you touch the cloud.
4. Push your image to a registry: **Amazon ECR** (AWS) or **Google Artifact Registry** (GCP).
5. Deploy the container (pick your cloud — see the two paths below).
6. Add an **Evidently** drift report comparing recent inputs to training data; surface it as a report or dashboard panel.
7. Document how to **roll back**: keep the previous image tag/version and state how you'd redeploy it.

GCP PATH

Google Cloud

- **Cloud Run** — deploy the container directly; scales to zero; has an always-free monthly request allowance. Best default for the API.
- **Artifact Registry** for the image; **Cloud Build** to build it.
- **Cloud Monitoring + Logging** for service metrics & logs.
- **Vertex AI** (optional, advanced) — managed Model Registry + built-in model monitoring for drift if you want to go beyond Evidently.
- New accounts get **$300 in credits for 90 days.**

AWS PATH

Amazon Web Services

- **Amazon ECS on Fargate** (use **ECS Express Mode** for the simplest setup) — run the container serverlessly. This is the current recommended path.
- **AWS Lambda** (container image) — good for pay-per-request, event-style inference.
- **Amazon ECR** for the image registry.
- **Amazon CloudWatch** for metrics, logs & alarms; **SageMaker Model Monitor** (optional, advanced) for managed drift detection.
- 12-month free tier + always-free allowances on several of these.

HEADS-UP — DON'T USE APP RUNNER

AWS App Runner is closed to new customers as of April 30, 2026. Use **ECS on Fargate (Express Mode)** or **Lambda** instead. (And note: we are **not** using Hugging Face Spaces for this project — you already covered Gradio in class.)

FREE-TIER NOTE

Every service above has a free option or free allowance, but limits and terms change — verify current limits on each provider's pricing page before relying on them. Set a billing alert. **Never commit secrets, keys, or real credentials to your repo.**

06 — WHAT YOU HAND IN

Not a notebook. A *system.*

- **A Git repository** with a README that reproduces everything from raw data to running service.
- **A reproducible data + training pipeline** (scripts, not just notebooks) with version tracking.
- **Baseline + advanced models** compared in MLflow, right metrics, justified threshold.
- **A Dockerized FastAPI service** + docker-compose, and a live (or one-command-deployable) endpoint.
- **An observability setup** — Prometheus metrics, Grafana dashboard, Evidently drift report — with a retraining trigger.
- **Governance artefacts** — fairness audit, SHAP explanations, audit-log sample, completed model card.
- **A 1–2 page reflection** — key trade-offs, what you'd change in production, the limits of your model.

What "done" looks like: someone clones your repo, follows the README, rebuilds the data, retrains the model, launches the service in one or two commands, hits the endpoint and gets a sensible risk score with explanations, opens the dashboard, and reads your model card to understand the limits — **without asking you anything.**